TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

INTRODUCTION TO PROLOG
Lab 1 Report

Submitted By:

Name: **Chandan Kumar Shah**

Roll No: **080BCT023**

Date: **2082-02-25**

Submitted To:

Department of Electronics
and Computer Engineering
Institute of Engineering,
Pulchowk Campus

Subject: Artificial Intelligence

Contents

1 Objectives

The primary objectives of this laboratory exercise are:

1. To revise and strengthen the fundamental concepts of predicates and clauses in logic programming.
2. To understand the declarative nature of the Prolog programming language and its application in artificial intelligence.
3. To implement various logical programs in Prolog and compare them with procedural (C++) equivalents.
4. To gain hands-on experience with Prolog's unification, backtracking, and recursion mechanisms.
5. To appreciate the differences between declarative and imperative programming paradigms.

2 Theoretical Background

2.1 What is Prolog?

Prolog (**Pro**grammation en **Log**ique) is a declarative, logic-based programming language that plays a pivotal role in artificial intelligence (AI) and computational linguistics. Unlike procedural languages such as C, C++, or Java, Prolog is based on formal logic—specifically, a subset of first-order predicate calculus known as Horn clauses. It was originally developed in 1972 by Alain Colmerauer and Philippe Roussel at the University of Aix-Marseille in France.

Prolog operates on the principle of *what* to compute rather than *how* to compute. The programmer defines a set of facts and rules that describe relationships and constraints, and the Prolog inference engine uses logical deduction and backtracking to derive answers to user queries. This declarative paradigm makes Prolog particularly well-suited for applications involving:

- Expert systems and knowledge representation
- Natural language processing (NLP)
- Automated theorem proving
- Database query languages
- Artificial intelligence and symbolic computation
- Constraint satisfaction problems

2.2 Fundamental Concepts in Prolog

2.2.1 Atoms

Atoms are basic constants that represent specific entities. They are strings of letters, digits, and underscores that begin with a lowercase letter.

- Examples: `john`, `parent`, `comp_students`
- Atoms enclosed in single quotes can start with uppercase: `'Ram'`, `'Bird'`
- Special character strings: `<->`, `::::::::::::`

2.2.2 Numbers

Prolog supports both integers and real (floating-point) numbers:

- Integers: `42`, `-7`, `0`
- Real numbers: `3.14`, `-2.5`, `1.0e10`

2.2.3 Variables

Variables represent unknown or placeholder values. They begin with an uppercase letter or an underscore:

- Examples: `X`, `Result`, `_weight`
- The scope of a variable is limited to a single clause.
- The anonymous variable `_` is used when a value is not needed elsewhere.

2.2.4 Facts

Facts are unconditionally true statements about the domain. They assert relationships between objects.

```
parent(john, mary).
parent(john, tom).
male(john).
female(mary).
```

Listing 1: Example facts

2.2.5 Rules

Rules define relationships conditionally, based on other facts and rules. They consist of a *head* (conclusion) and a *body* (premises), connected by the `:-` operator (read as "if").

```
father(F, C) :- parent(F, C), male(F).
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.
```

Listing 2: Example rules

2.2.6 Queries

Queries are questions posed to the Prolog system to determine whether a statement is true or to find values that satisfy given conditions.

```
?- father(john, mary).
% Output: true

?- grandparent(john, anna).
% Output: true
```

Listing 3: Example queries

2.3 List Structure in Prolog

Lists are one of the most important data structures in Prolog. A list is enclosed in square brackets and can contain any Prolog terms:

```
% A simple list
[ram, shyam, hari, sita]

% Head and Tail decomposition
[H | T]
% H = ram,  T = [shyam, hari, sita]

% Nested structure
[ram | [shyam, hari, sita]]
```

Listing 4: List structure

Lists in Prolog are internally represented as binary trees, where the head is the first element and the tail is the remaining list. This recursive structure enables elegant pattern matching and recursive processing.

2.4 Key Operators

Table 1: Key Prolog Operators

Operator	**Symbol**	**Meaning**
Conjunction	,	AND
Disjunction	;	OR
Unification	=	Pattern matching
Not equal	\=, <»	Inequality
Anonymous	_	Don't care variable
Cut	!	Prevent backtracking

2.5 Prolog vs. Procedural Programming

Table 2: Comparison: Prolog vs. Procedural Languages

Prolog (Declarative)	**C/C++ (Procedural)**
Defines *what* the problem is	Defines *how* to solve it
Uses facts and rules	Uses statements and instructions
Inference engine handles execution flow	Programmer controls flow
Backtracking explores alternatives	Explicit loops and conditionals
Pattern matching via unification	Explicit variable comparison
Recursive decomposition	Iterative or recursive functions

3 Program Implementations and Discussion

3.1 Program 1: HCF (GCD) of Two Numbers

3.1.1 Problem Statement

Find the Highest Common Factor (HCF) of two numbers using the Euclidean algorithm implemented in Prolog.

3.1.2 Mathematical Basis

The Euclidean algorithm is based on the principle that the HCF of two numbers also divides their difference. More formally:

$$\text{HCF}(X, Y) = \begin{cases} X & \text{if } Y = 0 \\ \text{HCF}(Y, X \bmod Y) & \text{if } Y > 0 \end{cases}$$

3.1.3 Prolog Code

```
% Assignment 1: HCF (GCD) of two numbers
% Usage: hcf(36, 24, R). -> R = 12

hcf(X, 0, X) :- X > 0.
hcf(X, Y, R) :-
    Y > 0,
    Rem is X mod Y,
    hcf(Y, Rem, R).

hcf(0, Y, Y) :- Y > 0.
```

Listing 5: HCF computation in Prolog

3.1.4 C++ Equivalent

```
#include <iostream>
using namespace std;

int hcf(int x, int y) {
    if (y == 0) return x;
    int r = x % y;
    return hcf(y, r);
}

int main() {
    int result = hcf(24, 36);
    cout << "HCF = " << result << endl;
    return 0;
}
```

Listing 6: HCF computation in C++

3.1.5 Output

```
?- hcf(24, 36, H).
HCF(24, 36) = 12
```

Listing 7: Program 1 Output

3.1.6 Discussion

The Prolog program defines the logical rules required to compute the HCF. The first rule states that if the second number becomes zero, the first number is the HCF (base case). The second rule reduces the problem by replacing the pair (X, Y) with $(Y, X \bmod Y)$ and recursing. The Prolog interpreter automatically applies the appropriate rule and continues the recursion until the base case is reached. In contrast, the C++ equivalent requires explicit function calls, conditional statements, and return values.

3.2 Program 2: Family Relationships (Facts, Rules, and Deductions)

3.2.1 Problem Statement

Model family relationships using Prolog facts and rules, and deduce various relationships through queries.

3.2.2 Prolog Code

```
% Facts
parent(john, mary).
parent(john, tom).
parent(mary, anna).
parent(mary, peter).
parent(tom, lisa).
parent(anna, david).

male(john). male(tom). male(peter). male(david).
female(mary). female(anna). female(lisa).

% Rules (deductions)
father(F, C) :- parent(F, C), male(F).
mother(M, C) :- parent(M, C), female(M).
grandparent(GP, GC) :- parent(GP, P), parent(P, GC).
sibling(A, B) :- parent(P, A), parent(P, B), A \= B.
ancestor(A, D) :- parent(A, D).
ancestor(A, D) :- parent(A, X), ancestor(X, D).
```

Listing 8: Family relationships in Prolog

3.2.3 C++ Equivalent

```cpp
#include <iostream>
using namespace std;

int main() {
    bool ramFootball = true, shyamFootball = true;
    if (ramFootball && shyamFootball)
        cout << "Ram and Shyam are friends" << endl;
    return 0;
}
```

Listing 9: Family relationships in C++

3.2.4 Output

```
?- father(john, mary).
true

?- grandparent(john, anna).
true

?- sibling(mary, tom).
true

?- ancestor(john, david).
true
```

Listing 10: Program 2 Output

3.2.5 Discussion

In Prolog, facts and rules declaratively encode family relationships. The inference engine automatically deduces relationships through unification and backtracking. For example, querying `grandparent(john, anna)` causes Prolog to chain the rules: `parent(john, mary)`, `parent(mary, anna)` → `true`. In C++, such relational reasoning requires explicit data structures and procedural logic.

3.3 Program 3: Sum of an Integer List

3.3.1 Problem Statement

Compute the sum of all elements in an integer list using recursive decomposition.

3.3.2 Mathematical Basis

The sum of a list can be defined recursively:

$$\text{sum\_list}([]) = 0$$

$$\text{sum\_list}([H|T]) = H + \text{sum\_list}(T)$$

3.3.3 Prolog Code

```
% Usage: sum_list([1, 2, 3, 4], S). -> S = 10

sum_list([], 0).
sum_list([H | T], Sum) :-
    sum_list(T, TailSum),
    Sum is H + TailSum.
```

Listing 11: Sum of list in Prolog

3.3.4 C++ Equivalent

```
#include <iostream>
using namespace std;

int main() {
    int a[] = {1, 2, 3, 4, 5};
    int sum = 0;
    for (int i = 0; i < 5; i++) sum += a[i];
    cout << "Sum = " << sum << endl;
    return 0;
}
```

Listing 12: Sum of list in C++

3.3.5 Output

```
?- sum_list([1, 2, 3, 4, 5], S).
S = 15
```

Listing 13: Program 3 Output

3.3.6 Discussion

The Prolog program defines the mathematical relationship between a list and its sum using recursion. The base case returns 0 for an empty list, and the recursive case adds the head element to the sum of the tail. The interpreter automatically evaluates the recursive rule through backtracking. In C++, a `for` loop is explicitly written to traverse the array and accumulate the sum.

3.4 Program 4: Length of a List

3.4.1 Problem Statement

Determine the number of elements in a list using recursive counting.

3.4.2 Mathematical Basis

$$\text{list\_length}([]) = 0$$
$$\text{list\_length}([\_|T]) = 1 + \text{list\_length}(T)$$

3.4.3 Prolog Code

```
% Usage: list_length([a, b, c, d], L). -> L = 4

list_length([], 0).
list_length([_ | T], Len) :-
    list_length(T, TailLen),
    Len is TailLen + 1.
```

Listing 14: List length in Prolog

3.4.4 C++ Equivalent

```
#include <iostream>
using namespace std;

int main() {
    int a[] = {1, 2, 3, 4, 5};
    cout << "Length = "
         << sizeof(a) / sizeof(a[0]) << endl;
    return 0;
}
```

Listing 15: List length in C++

3.4.5 Output

```
?- list_length([1, 2, 3, 4, 5], L).
L = 5
```

Listing 16: Program 4 Output

3.4.6 Discussion

In Prolog, list length is obtained recursively by counting elements until the empty list (base case) is reached. The anonymous variable `_` in the head position indicates we do not need the value of individual elements—only the count. In C++, the programmer directly calculates the size of the array using `sizeof` or iterates through it with a counter.

3.5 Program 5: Append Two Lists

3.5.1 Problem Statement

Concatenate two lists into a single list using recursive construction.

3.5.2 Mathematical Basis

$$\text{append\_lists}([], L_2) = L_2$$

$$\text{append\_lists}([H|T], L_2) = [H|\text{append\_lists}(T, L_2)]$$

3.5.3 Prolog Code

```
% Usage: append_lists([1, 2], [3, 4], R).
%        -> R = [1, 2, 3, 4]

append_lists([], L, L).
append_lists([H | T], L2, [H | Result]) :-
    append_lists(T, L2, Result).
```

Listing 17: List appending in Prolog

3.5.4 C++ Equivalent

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> a = {1, 2, 3};
    vector<int> b = {4, 5, 6};
    a.insert(a.end(), b.begin(), b.end());
    for (int x : a) cout << x << " ";
    cout << endl;
    return 0;
}
```

Listing 18: List appending in C++

3.5.5 Output

```
?- append_lists([1, 2, 3], [4, 5, 6], R).
R = [1, 2, 3, 4, 5, 6]
```

Listing 19: Program 5 Output

3.5.6 Discussion

The Prolog solution recursively constructs a new list by attaching elements from the first list before the second list. The base case is trivial: appending an empty list to any list yields that list. The recursive case takes the head of the first list and places it before the result of appending the tail. In C++, list concatenation is performed using vector operations and explicit library functions.

3.6 Program 6: Filter a List (Keep Only 1s and 2s)

3.6.1 Problem Statement

Given a list of integers, filter out all elements except those equal to 1 or 2.

3.6.2 Prolog Code

```
% Usage: filter_ones_twos([1,2,4,5,2,4,5,1,1], R).
%        -> R = [1, 2, 2, 1, 1]

filter_ones_twos([], []).
filter_ones_twos([H | T], [H | R]) :-
    (H = 1 ; H = 2),
    filter_ones_twos(T, R).
filter_ones_twos([H | T], R) :-
    H \= 1,
    H \= 2,
    filter_ones_twos(T, R).
```

Listing 20: Filtering in Prolog

3.6.3 C++ Equivalent

```
#include <iostream>
using namespace std;

int main() {
    int a[] = {1, 2, 4, 5, 2, 4, 5, 1, 1};
    for (int x : a)
        if (x == 1 || x == 2)
            cout << x << " ";
    cout << endl;
    return 0;
}
```

Listing 21: Filtering in C++

3.6.4 Output

```
?- filter_ones_twos([1,2,4,5,2,4,5,1,1], R).
R = [1, 2, 2, 1, 1]
```

Listing 22: Program 6 Output

3.6.5 Discussion

The Prolog program filters elements through logical rules and pattern matching. Values other than 1 and 2 are discarded automatically by the appropriate rule. The disjunction `(H = 1 ; H = 2)` in the second clause allows either match. In C++, each element is explicitly checked using conditional statements. Prolog performs declarative filtering, while C++ performs procedural filtering.

3.7 Program 7: Delete an Element from a List

3.7.1 Problem Statement

Remove all occurrences of a specified element from a list using recursive pattern matching.

3.7.2 Prolog Code

```
% Usage: delete_element(3, [1,2,3,4,3,5], R).
%        -> R = [1,2,4,5]

delete_element(_, [], []).
delete_element(X, [X | T], R) :- !, delete_element(X, T, R).
delete_element(X, [H | T], [H | R]) :- delete_element(X, T, R).
```

Listing 23: Element deletion in Prolog

3.7.3 C++ Equivalent

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 3, 5};
    vector<int> r;
    for (int x : v)
        if (x != 3) r.push_back(x);
    for (int x : r) cout << x << " ";
    cout << endl;
    return 0;
}
```

Listing 24: Element deletion in C++

3.7.4 Output

```
?- delete_element(3, [1, 2, 3, 4, 3, 5], R).
R = [1, 2, 4, 5]
```

Listing 25: Program 7 Output

3.7.5 Discussion

In Prolog, deletion is achieved by recursively rebuilding the list while skipping matching elements. The cut operator (`!`) in the second clause prevents backtracking, ensuring efficiency. The logic specifies what the resulting list should contain, not how to construct it. In C++, the programmer explicitly iterates through the collection and copies only the required elements.

4 Summary of Prolog vs. C++ Implementations

Table 3: Comparative Summary of All Programs

Program	**Prolog Approach**	**C++ Approach**
HCF	Recursive Euclidean algorithm via clauses	Recursive function with explicit modulo
Family Relationships	Declarative facts and rules with inference engine	Explicit data structures and conditional checks
Sum of List	Recursive decomposition: head + tail sum	Iterative loop with accumulator
List Length	Recursive counting with anonymous variable	`sizeof` operator or loop counter
Append Lists	Recursive list construction via pattern matching	Vector `insert` operation
Filter 1s & 2s	Pattern matching with disjunctive rules	Conditional `if` inside loop
Delete Element	Recursive rebuild with cut operator	Loop with conditional push_back

5 Conclusion

This laboratory exercise provided comprehensive hands-on experience with the Prolog programming language and its fundamental concepts. Through the implementation of seven distinct programs, we explored:

- **Declarative programming paradigm:** Prolog programs define relationships and constraints rather than step-by-step instructions. The Prolog inference engine handles execution flow through unification and backtracking.
- **Recursive decomposition:** All list-based programs (sum, length, append, filter, delete) demonstrate the elegance of recursive thinking in Prolog, where the base case and recursive case naturally express the problem.
- **Pattern matching:** Prolog's pattern matching through unification provides a powerful mechanism for data decomposition, particularly evident in list processing.
- **Knowledge representation:** The family relationships program demonstrates how Prolog can model complex relational knowledge through simple facts and rules.
- **Comparison with procedural languages:** The contrast between Prolog and C++ implementations highlights the fundamental differences between declarative and imperative programming paradigms.

The rule-based predicates and clauses demonstrate the declarative nature of Prolog. Here, we do not define explicit loops; rather, the rules are recursively implemented and terminate upon encountering the base case. The use of logic rules allows the generation of

new facts from the given facts. While Prolog focuses on describing relationships and logical conditions, C++ requires explicit control over the execution flow through loops, conditions, and functions.

Through this lab, we gained practical experience in Prolog programming and developed a better understanding of how logic programming differs from conventional programming approaches.

6 References

1. Bratko, I. (2012). *Prolog Programming for Artificial Intelligence* (4th ed.). Pearson Education.
2. Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog* (5th ed.). Springer-Verlag.
3. Sterling, L., & Shapiro, E. (1994). *The Art of Prolog: Advanced Programming Techniques* (2nd ed.). MIT Press.
4. Nilsson, U., & Maluszynski, J. (1995). *Logic, Programming and Prolog* (2nd ed.). John Wiley & Sons.
5. SWI-Prolog Documentation. Available at: `https://www.swi-prolog.org/pldoc/`